

Problem description

When the king of ghosts notices that all humans on Planet Earth have lost their dread of the ghost race, he is extremely unhappy. He understands why this is happening. The existing ghost species has gotten incredibly lethargic and has ceased visiting Planet Earth in order to terrorise humanity. As a result, he plans to arrange a tournament to urge the whole ghost race to scare the humans. The monarch, on the other hand, never comes to Planet Earth.

This competition will go on for **N days**. Currently, there are a total of **M ghosts** (apart from the king) existing in the ghost race such that :

- The youngest ghost is 1 year old.
- The oldest ghost is M years old.
- No two ghosts have the same age.
- The age of each and every ghost is a positive integer.

Every day of the tournament, ghosts must visit Planet Earth in order to frighten humans. The ghost that terrifies the most humans on any given day is named "Ghost of the Day" at the conclusion of the day. The king of ghosts, on the other hand, is a firm believer in constancy. After this title is granted, a "Consistency Trophy" is offered to the ghost who has earned the most of these titles up until that point. If there are several such ghosts, the prize is handed to the eldest among them. It's worth noting that "Title Giving" and "Trophy Giving" take place at the end of each tournament day.

Each day of the competition, you will be told the age of the ghost that earned the "Ghost of the Day" title. Your task is to determine the age of the ghost who received the "Consistency Trophy" on each competition day.

Input

The first line consists of 2 space separated integers **N** and **M**. The next line consists of **N** space separated integers such that the i^{th} integer denotes the age of the ghost who was awarded with the **"Ghost of the Day" title** on the i^{th} day of the competition.

Output

Print **N** lines. The i^{th} line should contain 2 space separated integers such that the first integer denotes the **age of the ghost** who was awarded with the

"Consistency Trophy" on the i^{th} day and the second integer denotes the number of "Ghost of the Day" titles won by this ghost until the end of the i^{th} day of the competition.

Constraints

$1 \leq N \leq 10^5$

$1 \leq M \leq 10^9$

Logical Test Cases

Test Case 1

INPUT (STDIN)

7 5
1 3 1 3 2 2 2

EXPECTED OUTPUT

1 1
3 1
1 2
3 2
3 2
3 2
2 3

Test Case 2

INPUT (STDIN)

10 4
3 4 5 1 3 1 3 2 2 2

EXPECTED OUTPUT

3 1
4 1
5 1
5 1
3 2
3 2
3 3
3 3
3 3
3 3

Mandatory Test Cases

Test Case 1

KEYWORD

for(i = 0;i<n-1;i++)

▼ Complexity Test Cases

Test Case 1

CYCLOMATIC COMPLEXITY

5

Test Case 2

TOKEN COUNT

190

Test Case 3

NLOC

28

Tree28.c +

```
1
2 #include <stdio.h>
3 #include <stdlib.h>
4
5 #define MAX_GHOST 100005
6
7 typedef struct {
8     int count;
9     int age;
10} HeapNode;
11
12typedef struct {
13     HeapNode *data;
14     int size;
15} MaxHeap;
16
17void push_heap(MaxHeap *h, int count, int age) {
18     int i = h->size++;
19     h->data[i].count = count;
20     h->data[i].age = age;
21     while (i > 0) {
22         int p = (i - 1) / 2;
23         int better = 0;
24         if (h->data[i].count > h->data[p].count) better = 1;
25         else if (h->data[i].count == h->data[p].count && h->data[i].age > h->data[p].age) better = 1;
26         if (better) {
27             HeapNode tmp = h->data[i];
28             h->data[i] = h->data[p];
29             h->data[p] = tmp;
30             i = p;
31         } else {
32             break;
33         }
34     }
35 }
```

```

32     }
33 }
34 }
35}
36
37void pop_heap(MaxHeap *h) {
38    h->data[0] = h->data[--h->size];
39    int i = 0;
40    while (2 * i + 1 < h->size) {
41        int left = 2 * i + 1;
42        int right = 2 * i + 2;
43        int best = left;
44        if (right < h->size) {
45            int right_better = 0;
46            if (h->data[right].count > h->data[left].count) right_better = 1;
47            else if (h->data[right].count == h->data[left].count && h->data[right].age > h->data[left].age) right_better = 1;
48            if (right_better) best = right;
49        }
50        int best_better = 0;
51        if (h->data[best].count > h->data[i].count) best_better = 1;
52        else if (h->data[best].count == h->data[i].count && h->data[best].age > h->data[i].age) best_better = 1;
53        if (best_better) {
54            HeapNode tmp = h->data[i];
55            h->data[i] = h->data[best];
56            h->data[best] = tmp;
57        }
58    }
59}

```

```

57         i = best;
58     } else {
59         break;
60     }
61 }
62}
63
64int count_arr[MAX_GHOST];
65
66int main() {
67     int n, m;
68     if (scanf("%d %d", &n, &m) != 2) return 0;
69
70     int *ages = (int*)malloc(n * sizeof(int));
71     int i;
72     for (i = 0; i < n; i++) {
73         if (scanf("%d", &ages[i]) != 1) ages[i] = 0;
74     }
75
76     MaxHeap h;
77     h.data = (HeapNode*)malloc((n + 5) * sizeof(HeapNode));
78     h.size = 0;
79
80     for(i = 0; i < n - 1; i++) {
81         int age = ages[i];

```

```

82     count_arr[age]++;
83     push_heap(&h, count_arr[age], age);
84     while (h.size > 0 && h.data[0].count != count_arr[h.data[0].age]) {
85         pop_heap(&h);
86     }
87     printf("%d %d\n", h.data[0].age, h.data[0].count);
88 }
89 if (n > 0) {
90     int age = ages[n - 1];
91     count_arr[age]++;
92     push_heap(&h, count_arr[age], age);
93     while (h.size > 0 && h.data[0].count != count_arr[h.data[0].age]) {
94         pop_heap(&h);
95     }
96     printf("%d %d\n", h.data[0].age, h.data[0].count);
97 }
98
99 free(ages);
100 free(h.data);
101 return 0;
102
103

```



```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t1.txt
```

```
7 5
```

```
1 3 1 3 2 2 2
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Tree28.exe
```

```
1 1
```

```
3 1
```

```
1 2
```

```
3 2
```

```
3 2
```

```
3 2
```

```
2 3
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t2.txt
```

```
10 4
```

```
3 4 5 1 3 1 3 2 2 2
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Tree28.exe
```

```
3 1
```

```
4 1
```

```
5 1
```

```
5 1
```

```
3 2
```

```
3 2
```

```
3 3
```

```
3 3
```

```
3 3
```

```
3 3
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>
```